Humain Hackathon Research

Agentic Diagnosis and Recovery of Failing Systems:
A Structured Literature Review of AIOps, LLM Agents, and Security-Aware Operations

Abdulrahman Alharbi

October 2026

Abstract

Modern enterprise systems fail in ways that are hard to observe and harder to untangle: faults propagate across service dependencies, several issues can occur at once, and security incidents can masquerade as ordinary performance degradation. This review surveys the state of research on automated failure management, tracing its path from autonomic computing and classical machine-learning root cause analysis (RCA) to large language model (LLM) agents that diagnose and mitigate incidents end-to-end. We organize the literature into five threads: (i) conceptual foundations, (ii) classical anomaly detection and RCA, (iii) LLM-assisted incident management, (iv) agentic site reliability engineering (SRE) and its benchmarks, and (v) safety of autonomous remediation, with a complementary thread on periodicity-based detection of command-and-control (C2) beaconing. The evidence shows a field in rapid transition: LLM agents are now evaluated in live, fault-injected environments, yet reported success rates on realistic benchmarks remain low, and evaluations rarely address concurrent faults, verification of recovery, the operational cost of the agent itself, or the overlap between operational and security incidents. We conclude with a gap analysis and derive the research questions and evaluation protocol for a hackathon prototype targeting Challenge 1 (*Agentic Recovery of a Failing System*).

1 Introduction

Incident management is one of the most expensive and least automated parts of operating software. On-call engineers must detect a problem, localize it, determine its root cause, mitigate it, and confirm that the system is healthy again, often under time pressure and with incomplete information. The literature on *AI for IT Operations* (AIOps) has attacked pieces of this lifecycle for more than a decade [14, 18]. Since 2023, the arrival of capable LLMs has shifted the research question from "can a model rank likely root causes?" to "can an agent own the incident end-to-end?" [1, 5, 17].

This review asks four questions:

RQ1. What methods exist for automated detection, diagnosis, and recovery of failing distributed systems, and how have they evolved?

RQ2. How are these methods evaluated, and what benchmarks define the current state of the art?

RQ3. What is known about the safety and reliability of agents that take corrective action autonomously?

RQ4. Which open problems remain, and which of them can be credibly addressed in a time-boxed prototype?

2 Review Methodology

Review type. This is a *structured narrative literature review*, not a PRISMA-style systematic review or a meta-analysis. A formal meta-analysis is not appropriate here: the primary studies use different systems, fault taxonomies, and metrics (AC@k, Micro-F1, mitigation success rate, semantic similarity), so effect sizes cannot be pooled meaningfully. Instead, we report results side-by-side with their evaluation context and synthesize qualitatively.

Sources and inclusion criteria. We prioritized peer-reviewed venues: IEEE (*Computer*, *Software*, NOMS, DSN), ACM (*Computing Surveys*, *TIST*, EuroSys, ASPLOS, CIKM, SoCC, WWW), and the main machine-learning conferences (ICLR, ICML, NeurIPS, MLSys), plus the Elsevier journal *Computers & Security*. Works were included if they (a) propose a method for anomaly detection, RCA, or mitigation in distributed or cloud systems; (b) propose a benchmark or evaluation framework for such methods; (c) study the safety of tool-using LLM agents; or (d) address periodicity-based detection of malicious network communication. One recent preprint (an LLM-era AIOps survey [23]) is included because it is the most complete map of the post-2023 literature; it is clearly marked as a preprint.

Limitations of the method. The search was time-boxed to the hackathon setting, so coverage is representative rather than exhaustive; Section 12 discusses threats to validity.

3 Foundations: From Autonomic Computing to AIOps

The goal of self-healing systems predates modern machine learning. Kephart and Chess [13] articulated the *autonomic computing* vision in which systems manage themselves according to high-level objectives set by administrators. Their architecture, later popularized as the Monitor–Analyze–Plan–Execute loop over shared knowledge (MAPE-K), remains the conceptual skeleton of today's agentic designs: telemetry is monitored, analyzed for symptoms, a plan is formed, and actions are executed and observed.

Two engineering disciplines supply the practical vocabulary. Site reliability engineering, codified at Google [4], frames reliability in terms of service-level objectives, error budgets, and mean time to recovery (MTTR). Chaos engineering, introduced at Netflix [3], treats resilience as an experimental science: one hypothesizes a steady state, injects real-world faults, and measures deviation. Chaos engineering is directly relevant to evaluation: deliberate fault injection is the standard way to obtain ground truth for diagnosis and recovery experiments.

Two surveys map the pre-LLM AIOps landscape. Notaro et al. [14] organize AIOps for failure management into categories by intervention time (failure prevention, prediction, detection, root cause analysis, and remediation) and note that remediation is the least mature. Soldani and Brogi [18] focus on multi-service applications and observe that anomaly detection and RCA techniques are often developed in isolation, leaving operators to stitch them into a working pipeline.

4 Classical Anomaly Detection and Root Cause Analysis

Before LLMs, RCA for microservices was dominated by graph-based and causal methods operating on metrics and traces. MicroRCA [20] builds an attributed graph linking services and hosts, correlates application-level latency symptoms with resource utilization, and ranks culprit services

via graph centrality; on fault injections into a Kubernetes-hosted benchmark it reports 89% precision and 97% mean average precision. Sage [9] uses unsupervised causal Bayesian networks and a graphical variational autoencoder to reason counterfactually about which microservice caused a quality-of-service violation, avoiding the need for labeled traces, and reports identifying the correct root cause in over 93% of cases. Sage is notable because it also *acts*: it applies corrective resource adjustments, closing the loop that most RCA work leaves open. Both rely on standard microservice testbeds such as DeathStarBench [8].

These methods are fast, cheap, and interpretable, but they share three limitations that motivate LLM-based work: they target a fixed fault taxonomy (usually resource contention and latency), they produce a ranking rather than an explanation or a remediation plan, and they generally assume a single root cause per incident.

5 LLM-Assisted Incident Management

Recommendation from incident text. Ahmed et al. [1] conducted the first large-scale study of LLMs for cloud incident RCA and mitigation, evaluating GPT-3.x models in zero-shot, fine-tuned, and multi-task settings on more than 40,000 Microsoft incidents. Their results established feasibility, and their human evaluation with incident owners showed practical value, but the models consumed only incident titles and summaries; they could not gather new evidence.

Workflow-driven diagnosis. RCACopilot [5] separates the problem into two stages: incident-specific handlers collect multi-source diagnostic data, and an LLM predicts the root-cause category with an explanation, using similar historical incidents as context. Evaluated on a year of Microsoft incidents, it achieved RCA accuracy up to 0.766 (Micro-F1), and its data-collection component has been used in production for years. The design principle that *deterministic tooling gathers evidence and the LLM reasons over it* is important for this project.

Tool-augmented autonomous agents. RCAgent [19] moved to the agent paradigm: an LLM controller follows a thought–action–observation loop in the style of ReAct [22], calls information-gathering and expert-analysis tools, and aggregates trajectories with self-consistency. Two of its contributions are directly reusable. First, it targets an *internally deployed* model rather than an external API for privacy reasons, showing that agentic RCA does not require frontier proprietary models. Second, it introduces stabilization methods (structured output repair, context management) that sharply reduce invalid actions, a dominant failure mode of agents operating on noisy telemetry.

The post-2023 literature has grown quickly; Zhang et al. [23] survey it across data preprocessing, failure perception, RCA, and auto-remediation and conclude that remediation remains the least developed stage.

6 Agentic SRE and Its Benchmarks

A key development since 2024 is the move from static datasets to *interactive* evaluation environments in which agents act on a live system.

- **AIOpsLab** [17, 6] (Microsoft, SoCC'24 vision paper and MLSys'25) deploys microservice applications, injects faults, generates workload, exports telemetry, and exposes an *agent–cloud*

interface. Its benchmark of 48 problems spans detection, localization, RCA, and mitigation, and frames the long-term goal as *AgentOps*: agents that own the full incident lifecycle.

- **ITBench** [12] (IBM, ICML'25) covers SRE, compliance/security operations (CISO), and FinOps with push-button workflows and interpretable metrics. Its ICML release contains 102 scenarios; agents built on state-of-the-art models resolved only about 11% of SRE scenarios and about 25% of CISO scenarios.
- **OpenRCA** [21] (ICLR'25) provides 335 real failure cases from three enterprise systems with over 68 GB of logs, metrics, and traces. Even the best configuration on its public leaderboard (an RCA agent using Claude 3.5 Sonnet) identifies the full root cause in only about 11% of cases. The authors also report that models often reach correct answers through flawed reasoning, which argues for evaluating the reasoning path, not only the final label.
- **RCAEval** [15] (WWW'25 Companion) contributes 735 failure cases from three microservice systems covering 11 fault types (resource, network, and code-level) with fifteen reproducible baselines, standardizing ranking metrics such as AC@k.

State of the art in mitigation. STRATUS [7] (NeurIPS'25) organizes specialized detection, diagnosis, and mitigation agents in a state machine and formalizes a safety property called *Transactional No-Regression* (TNR): any mitigation attempt that does not improve the system can be rolled back, enabling safe exploration. It reports at least a 1.5× improvement in mitigation success over prior SRE agents on both AIOpsLab and ITBench, across several models.

Table 1: Representative evaluation frameworks and reported results. Figures are *not* directly comparable: systems, fault sets, and metrics differ.

Work	Venue	Setting	Headline result
MicroRCA [20]	IEEE NOMS'20	Graph RCA, injected faults, Kubernetes	89% precision, 97% MAP
Sage [9]	ASPLOS'21	Unsupervised causal RCA + actuation	>93% root cause identified
Ahmed et al. [1]	ICSE'23	40k+ incidents, text only	Feasibility; human-rated usefulness
RCACopilot [5]	EuroSys'24	Handlers + LLM, real incidents	Micro-F1 up to 0.766
RCAgent [19]	CIKM'24	Tool agent, internal model	Outperforms ReAct; fewer invalid actions
AIOpsLab [6]	MLSys'25	Live, fault-injected microservices	48 problems, 4 agent types
ITBench [12]	ICML'25	SRE / CISO / FinOps scenarios	≈11% SRE, ≈25% CISO solved
OpenRCA [21]	ICLR'25	335 real failures, 68 GB telemetry	Best ≈11% fully correct
RCAEval [15]	WWW'25	735 cases, 11 fault types	15 baselines, AC@k
STRATUS [7]	NeurIPS'25	Multi-agent SRE with rollback	≥1.5× mitigation success

7 Safety of Autonomous Remediation

An agent that can restart services, roll back deployments, or isolate hosts can also make an outage worse. ToolEmu [16] (ICLR'24) built an LM-emulated sandbox and an automatic safety evaluator to find risky agent behavior at scale; even the safest agent they tested still exhibited risky failures in 23.9% of test cases, and 68.8% of flagged failures were judged valid real-world failures by human

reviewers. In the operations domain specifically, STRATUS's TNR property [7] is the most rigorous treatment to date, but it addresses rollback of ineffective mitigations, not the question of whether a mitigation is *appropriate* given the incident's nature (for example, restarting a compromised container may destroy forensic evidence and allow re-infection).

8 The Security–Operations Boundary: C2 Beaconing

The AIOps literature models faults as accidental (resource exhaustion, latency, misconfiguration, code bugs). In practice, some "performance incidents" are security incidents. Malware that has established a foothold periodically contacts its command-and-control server, a behavior known as *beaconing*. BAYWATCH [11] (IEEE/IFIP DSN'16) detects stealthy beaconing in enterprise web-proxy logs via signal processing and an eight-step filtering pipeline, evaluated on more than 30 billion events from a corporate network of over 130,000 devices, and emphasizes that legitimate software (such as update checks) also beacons, making false positives the central challenge. The CTU-13 dataset [10] (*Computers & Security*) supplies labeled botnet, normal, and background traffic and remains a standard benchmark for this problem.

The author's prior work [2] showed that periodicity features computed from flow timestamps alone (inter-arrival regularity, FFT spectral concentration, autocorrelation, sub-window stability) can rank suspicious communication units on encrypted traffic, reaching an average precision of 0.852 against a heuristic baseline of 0.046 on CTU-13 with synthetic beacon injection. These features are cheap to compute and require no payload inspection, which makes them suitable as a deterministic detector inside an operations agent.

None of the AIOps benchmarks reviewed in Section 6 include a scenario in which a security incident co-occurs with, or is disguised as, an operational fault, although ITBench's CISO track addresses compliance-oriented security tasks separately [12].

9 Synthesis: Where the Field Stands

Across the reviewed work, five observations stand out.

1. **The paradigm has shifted to interactive agents.** The research frontier is no longer offline ranking but agents that gather evidence, act, and are evaluated in live fault-injected environments [6, 12, 7].

2. **Absolute performance on realistic tasks is low.** Benchmarks with real failures report success rates around 11% for RCA and SRE resolution [21, 12]. The problem is far from solved.

3. **Hybrid designs dominate in production.** Systems deployed in industry pair deterministic evidence collection with LLM reasoning [5, 19] rather than giving a model raw telemetry.

4. **Safety is recognized but immature.** Rollback-based guarantees exist [7], and general agent-risk studies show high rates of risky behavior [16], but action *appropriateness* remains unaddressed.

5. **Evaluation metrics are fragmented.** Studies use AC@k, Micro-F1, mitigation success, and human ratings, which prevents meta-analysis and makes progress hard to track.

10 Gap Analysis

G1. Concurrent and confounded faults. Most evaluations inject one fault per scenario, and classical RCA assumes a single root cause, although real incidents often involve several interacting issues.

G2. Security-blind operations. No reviewed AIOps benchmark evaluates whether an agent recognizes that an apparent operational fault is, or co-occurs with, a compromise, nor whether its remediation is appropriate in that case.

G3. Verification of recovery. Mitigation success is measured, but agents falsely declaring recovery is rarely reported as a distinct error class.

G4. Cost of the agent. Benchmarks seldom report tokens, latency, or compute per incident, and evaluations mostly use large proprietary models; RCAgent [19] is a notable exception in using an internally deployed model.

G5. Robustness and generalization. Few studies systematically test performance under telemetry degradation (missing metrics, noisy logs) or on held-out fault combinations.

11 Implications for This Project

The gap analysis suggests a contribution that is both novel and feasible in 48 hours: a *security-aware, hybrid diagnosis-and-recovery agent* evaluated with a transparent, reproducible benchmark. Following the design principle established by RCACopilot and RCAgent, deterministic detectors (metric anomaly detection, log pattern matching, and the periodicity features of [2]) produce a ranked evidence list, and a locally hosted LLM performs hypothesis formation, targeted checks, action selection from an allowlist, and recovery verification, structured as a MAPE-K loop [13].

The resulting research questions are:

P1. Does a hypothesis–test–verify agent identify all root causes in concurrent-fault scenarios more accurately than a rule-based runbook and a single-shot LLM? (G1)

P2. Can the agent detect a co-occurring C2 beacon and choose containment before restart? (G2)

P3. How often does each approach falsely claim recovery? (G3)

P4. What accuracy is lost, and what cost is saved, when the LLM reasons over pre-ranked evidence rather than raw telemetry? (G4)

P5. How much does performance degrade on held-out fault combinations and under telemetry perturbations? (G5)

Evaluation protocol. Following chaos-engineering practice [3] and the benchmark designs of AIOpsLab and RCAEval, scenarios are defined declaratively with injected ground truth and split into development and held-out sets. Metrics include Top-k root-cause accuracy (AC@k [15]), per-fault recall on multi-fault scenarios, recovery success rate, MTTR, false-recovery rate, unnecessary or harmful actions per incident, and tokens and wall-clock time per incident. Component contributions are measured by ablation (removing the verification step, the hypothesis loop, the security detector, and the safety layer), and a smoke subset runs as a regression suite in continuous integration.

12 Limitations of This Review

This review was conducted under hackathon time constraints. Coverage is representative rather than exhaustive, and the field moves quickly: several cited works appeared in 2025, and newer results may exist. Reported numbers are taken from the original papers and their public leaderboards and were not reproduced. Because metrics and settings differ across studies, Table 1 should be read as a map of the landscape, not as a ranking.

References

[1] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023.

[2] A. Alharbi, "Periodicity-based triage of APT command-and-control beaconing in network flows using machine learning," M.S. thesis, Dept. of Computer Science, University of Nevada, Las Vegas, 2026.

[3] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[4] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.

[5] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. on Computer Systems (EuroSys)*, 2024, doi: 10.1145/3627703.3629553.

[6] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," in *Proc. 8th Conf. on Machine Learning and Systems (MLSys)*, 2025.

[7] Y. Chen, J. Pan, J. Clark, Y. Su, N. Zheutlin, B. Bhavya, R. R. Arora, Y. Deng, S. Jha, and T. Xu, "STRATUS: A multi-agent system for autonomous reliability engineering of modern clouds," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025.

[8] Y. Gan *et al.*, "An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems," in *Proc. 24th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.

[9] Y. Gan, M. Liang, S. Dev, D. Lo, and C. Delimitrou, "Sage: Practical and scalable ML-driven performance debugging in microservices," in *Proc. 26th Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021, doi: 10.1145/3445814.3446700.

[10] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Computers & Security*, vol. 45, pp. 100–123, 2014, doi: 10.1016/j.cose.2014.05.011.

[11] X. Hu, J. Jang, M. P. Stoecklin, T. Wang, D. L. Schales, D. Kirat, and J. R. Rao, "BAYWATCH: Robust beaconing detection to identify infected hosts in large-scale enterprise networks," in *Proc. 46th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2016, pp. 479–490.

[12] S. Jha *et al.*, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," in *Proc. 42nd Int. Conf. on Machine Learning (ICML)*, PMLR vol. 267, 2025, pp. 27134–27197.

[13] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *IEEE Computer*, vol. 36, no. 1, pp. 41–50, 2003, doi: 10.1109/MC.2003.1160055.

[14] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Trans. Intelligent Systems and Technology*, vol. 12, no. 6, art. 81, 2021, doi: 10.1145/3483424.

[15] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proc. ACM Web Conference (WWW Companion)*, 2025.

[16] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024.

[17] M. Shetty, Y. Chen, G. Somashekar, M. Ma, Y. Simmhan, X. Zhang, J. Mace, D. Vandevoorde, P. Las-Casas, S. M. Gupta, S. Nath, C. Bansal, and S. Rajmohan, "Building AI agents for autonomous clouds: Challenges and design principles," in *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2024, pp. 99–110.

[18] J. Soldani and A. Brogi, "Anomaly detection and failure root cause analysis in (micro)service-based cloud applications: A survey," *ACM Computing Surveys*, vol. 55, no. 3, pp. 1–39, 2022, doi: 10.1145/3501297.

[19] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proc. 33rd ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2024, doi: 10.1145/3627673.3680016.

[20] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root cause localization of performance issues in microservices," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020, pp. 1–9, doi: 10.1109/NOMS47738.2020.9110353.

[21] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "OpenRCA: Can large language models locate the root cause of software failures?" in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, 2025.

[22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. on Learning Representations (ICLR)*, 2023.

[23] L. Zhang, T. Jia, M. Jia, Y. Wu *et al.*, "A survey of AIOps for failure management in the era of large language models," arXiv preprint arXiv:2406.11213, 2024.